MCA SEM-I

SUBJECT: Open-Source Web Programming Using PHP

UNIT- II

Unit: Functions, Strings, Regular Expressions and Arrays

PART A — FUNCTIONS IN PHP

1. Introduction to Functions

A function is a named, reusable block of statements that performs a particular task.

Functions are one of the most important concepts in programming because they allow a large program to be divided into smaller, manageable modules.

Example

```
<?php
function displayMessage() {
   echo "Welcome to PHP Programming";
}
displayMessage();
?>
```

Output

Welcome to PHP Programming

The function displayMessage() is defined once and can be called whenever required.

Advantages of Functions

1. Code reusability
2. Reduced code duplication
3. Better program organization
4. Easier debugging
5. Easier maintenance
6. Improved readability
7. Supports modular programming

---

2. Defining a Function

A function is defined using the function keyword.

Syntax

```
function functionName(parameters) {
   // statements
}
```

Example

```
<?php
function greet() {
   echo "Hello Students";
}
?>
```

The above code defines the function but does not execute it.

To execute it, we need to call the function.

---

3. Calling a Function

Calling a function means requesting PHP to execute the statements contained inside the function.

Syntax

```
functionName();
```

Example

```
<?php
function greet() {
   echo "Hello Students";
}
greet();
?>
```

Output

Hello Students

Calling a Function Multiple Times

```
<?php
function message() {
   echo "PHP is easy to learn.<br>";
}
message();
message();
message();
```

ITM-MCA

```
?>
```

Output

```
PHP is easy to learn.
PHP is easy to learn.
PHP is easy to learn.
```

---

4. Function Parameters

A parameter is a variable declared in a function definition that receives a value when the function is called.

Example

```
<?php
function addition($a, $b) {
   echo $a + $b;
}
addition(10, 20);
?>
```

Output

```
30
```

Here:

- $a and $b → Parameters
- 10 and 20 → Arguments

Parameter vs Argument

Parameter	Argument
Defined in function declaration	Supplied during function call
Acts as a placeholder	Actual value
Example: $a	Example: 10

---

5. Multiple Parameters

A function can accept multiple parameters.

```
<?php
function studentDetails($name, $course, $age) {
   echo "Name: $name<br>";
   echo "Course: $course<br>";
   echo "Age: $age<br>";
}
```

ITM-MCA

```
studentDetails("Rahul", "MCA", 23);
?>
```

Output

```
Name: Rahul
Course: MCA
Age: 23
```

---

6. Default Parameters

A parameter can have a default value.

```
<?php
function greet($name = "Student") {
   echo "Hello " . $name;
}
greet();
echo "<br>";
greet("Amit");
?>
```

Output

```
Hello Student
Hello Amit
```

If an argument is not supplied, PHP uses the default value.

---

7. Passing Arguments by Value

By default, PHP passes arguments by value.

A copy of the variable is passed to the function.

```
<?php
function changeValue($x) {
   $x = 100;
}
$num = 10;
changeValue($num);
echo $num;
?>
```

Output

```
10
```

The original $num is not changed.

ITM-MCA

8. Passing Arguments by Reference

An argument can be passed by reference using &.

```
<?php
function changeValue(&$x) {
   $x = 100;
}
$num = 10;
changeValue($num);
echo $num;
?>
```

Output

100

The function modifies the original variable.

Difference

Pass by Value	Pass by Reference
Copy is passed	Reference is passed
Original variable normally remains unchanged	Original variable can be modified
No &	Uses &

---

9. Variable Scope

Variable scope defines the part of a program where a variable can be accessed.

Important scopes in PHP include:

1. Local scope
2. Global scope
3. Static variables
4. Superglobals

9.1 Local Scope

A variable declared inside a function has local scope.

```
<?php
function test() {
   $x = 10;
   echo $x;
}
test();
```

ITM-MCA

?>

$x is available inside test().

It cannot normally be accessed outside the function.

9.2 Global Scope

A variable declared outside a function has global scope.

```
<?php
$x = 100;
echo $x;
?>
```

To access a global variable inside a function, the global keyword can be used.

```
<?php
$x = 100;
function display() {
   global $x;
   echo $x;
}
display();
?>
```

Output

100

9.3 $GLOBALS

PHP provides the $GLOBALS array for accessing global variables.

```
<?php
$x = 100;
function display() {
   echo $GLOBALS['x'];
}
display();
?>
```

Output

100

9.4 Static Variables

A static variable retains its value between function calls.

```
<?php
```

ITM-MCA

```
function counter() {
   static $count = 0;
   $count++;
   echo $count . "<br>";
}
counter();
counter();
counter();
?>
```

Output

1

2

3

Without static, $count would be initialized again during each function call.

---

10. Return Values

The return statement sends a value from a function back to the calling code.

Syntax

```
return value;
```

Example

```
<?php
function add($a, $b) {
   return $a + $b;
}
$result = add(10, 20);
echo $result;
?>
```

Output

30

echo vs return

Echo:

```
function add($a, $b) {
   echo $a + $b;
}
```

Return:

```
function add($a, $b) {
```

IITM-MCA

```
    return $a + $b;
}
```

A returned value can be stored and reused:

```
$result = add(10, 20);
$total = $result + 50;
echo $total;
```

---

11. Variable Functions

A variable function allows a function to be called using a variable containing the function's name.

```
<?php
function greet() {
    echo "Hello MCA Students";
}
$functionName = "greet";
$functionName();
?>
```

Output

Hello MCA Students

Example

```
<?php
function add($a, $b) {
    return $a + $b;
}
function multiply($a, $b) {
    return $a * $b;
}
$operation = "multiply";
echo $operation(5, 4);
?>
```

Output

20

Applications

Variable functions can be useful in:

- Dynamic menu systems
- Callback mechanisms
- Dynamic operations

ITM-MCA

- Event handling
- Configurable applications

---

12. Anonymous Functions

An anonymous function is a function without a name.

It can be assigned to a variable.

Syntax

```
$variable = function(parameters) {
   // statements
};
```

Example

```
<?php
$greet = function() {
   echo "Hello MCA Students";
};
$greet();
?>
```

Output

Hello MCA Students

With Parameters

```
<?php
$add = function($a, $b) {
   return $a + $b;
};
echo $add(10, 20);
?>
```

Output

30

ITM-MCA

PART B — STRINGS IN PHP

13. Introduction to Strings

A string is a sequence of characters.

Examples:

$name = "Rahul";

$course = "MCA";

$message = "Welcome to PHP";

PHP supports both single and double quotes for strings.

---

14. Quoting String Constants

14.1 Single-Quoted Strings

Single quotes are written using ' '.

$name = 'Rahul';

PHP performs limited interpretation inside single quotes.

$name = 'Rahul';

echo 'Hello $name';

Output

Hello $name

The variable is not expanded.

14.2 Double-Quoted Strings

Double quotes are written using " ".

PHP can interpret variables and escape sequences inside double-quoted strings.

$name = "Rahul";

echo "Hello $name";

Output

Hello Rahul

Comparison

Single Quotes	Double Quotes
'Hello'	"Hello"
Variables are not normally interpolated	Variables can be interpolated
Fewer escape sequences interpreted	More escape sequences interpreted

---

15. Printing Strings

PHP provides several ways to display strings.

echo

ITM-MCA

echo "Hello PHP";

Multiple strings can be supplied:

echo "Hello ", "MCA ", "Students";

---

print

print "Hello PHP";

print returns a value and accepts one argument, while echo is commonly used for straightforward output.

---

printf()

printf() displays formatted strings.

$name = "Rahul";

$marks = 85;

printf("Name: %s, Marks: %d", $name, $marks);

Output

Name: Rahul, Marks: 85

Common format specifiers:

Specifier	Meaning
%s	String
%d	Integer
%f	Floating-point number
%x	Hexadecimal

---

16. Accessing Individual Characters

Individual characters of a string can be accessed using square brackets.

$text = "HELLO";

echo $text[0];

Output

H

Indexes start from 0.

Index	Character
0	H
1	E
2	L
3	L
4	O

ITM-MCA

Example

```
$text = "HELLO";
echo $text[1];
echo $text[3];
```

Output:

EL

A character can also be modified:

```
$text = "HELLO";
$text[0] = "Y";
echo $text;
```

Output:

YELLO

---

17. Cleaning Strings

When accepting input from users, strings may contain unwanted spaces or characters.

PHP provides several functions for cleaning strings.

trim()

Removes whitespace from the beginning and end.

```
$name = "   Rahul   ";
echo trim($name);
```

Output:

Rahul

---

ltrim()

Removes whitespace from the beginning.

```
$text = "   Hello";
echo ltrim($text);
```

---

rtrim()

Removes whitespace from the end.

```
$text = "Hello   ";
echo rtrim($text);
```

---

str_replace()

Used to replace characters or strings.

```
$text = "Hello World";
echo str_replace("World", "PHP", $text);
```

ITM-MCA

Output:

Hello PHP

Important Note

For user-submitted HTML content, "cleaning" and "security escaping" are different operations. Functions such as trim() remove unwanted whitespace; they do not make HTML or SQL input safe.

---

18. Encoding and Escaping

Encoding and escaping are important when strings are used with HTML, URLs, JSON, databases, or other systems.

HTML Escaping

htmlspecialchars() converts special HTML characters into safe HTML entities.

```
$text = "<h1>Hello</h1>";
echo htmlspecialchars($text);
```

Output:

<h1>Hello</h1>

This is especially important when displaying user-controlled text in an HTML page.

---

htmlspecialchars()

Common conversions include:

Character	HTML Entity
<	<
>	>
&	&
"	"

Example:

```
$name = "<script>alert('Hello');</script>";
echo htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
```

The browser displays it as text instead of interpreting it as HTML.

URL Encoding

urlencode() encodes a string for use in a URL query component.

```
$text = "PHP Programming";
echo urlencode($text);
```

A space becomes encoded appropriately for query-string usage.

ITM-MCA

19. Comparing Strings

PHP provides several methods for comparing strings.

strcmp()

Performs a case-sensitive comparison.

```
$a = "Hello";
$b = "Hello";
echo strcmp($a, $b);
```

Returns:

- 0 → strings are equal
- Negative value → first string sorts before second
- Positive value → first string sorts after second

---

strcasecmp()

Performs a case-insensitive comparison.

```
$a = "HELLO";
$b = "hello";
echo strcasecmp($a, $b);
```

Returns 0 because the strings differ only in case.

Checks whether two strings are identical in both value and type.

```
$a = "10";
$b = "10";
if ($a === $b) {
    echo "Strings are identical";
}
```

---

20. Manipulating Strings

PHP provides many built-in functions for string manipulation.

strlen()

Returns the length of a string.

```
$text = "Hello";
echo strlen($text);
```

Output:

5

---

strtolower()

Converts a string to lowercase.

ITM-MCA

echo strtolower("HELLO");

Output:

hello

---

strtoupper()

Converts a string to uppercase.

echo strtoupper("hello");

Output:

HELLO

---

ucfirst()

Converts the first character to uppercase.

echo ucfirst("hello");

Output:

Hello

---

substr()

Extracts part of a string.

$text = "Programming";

echo substr($text, 0, 7);

Output:

Program

---

strrev()

Reverses a string.

echo strrev("Hello");

Output:

olleH

---

explode()

Converts a string into an array.

$text = "MCA,PHP,Java";

$result = explode(",", $text);

print_r($result);

Output:

Array

(

ITM-MCA

```
    [0] => MCA
    [1] => PHP
    [2] => Java
)
```

---

implode()

Combines array elements into a string.

```
$data = ["MCA", "PHP", "Java"];
echo implode(",", $data);
```

Output:

```
MCA,PHP,Java
```

---

21. Searching Strings

strpos()

Finds the position of the first occurrence of a substring.

```
$text = "Welcome to PHP";
echo strpos($text, "PHP");
```

The returned position is zero-based.

Important

Always check strpos() using strict comparison because position 0 is a valid result.

```
if (strpos($text, "Welcome") !== false) {
    echo "Found";
}
```

---

str_contains()

Modern PHP provides str_contains() for checking whether a string contains another string.

```
$text = "Welcome to PHP";
if (str_contains($text, "PHP")) {
    echo "Found";
}
```

---

str_replace()

```
$text = "I like Java";
$result = str_replace("Java", "PHP", $text);
echo $result;
```

Output:

```
I like PHP
```

ITM-MCA

PART C — REGULAR EXPRESSIONS

22. Introduction to Regular Expressions

A regular expression (regex) is a pattern used to search, match, validate, extract, or replace text.

Regular expressions are useful for:

- Email validation
- Phone number validation
- Password validation
- Searching text
- Extracting data
- Replacing text
- Form validation

Example pattern:

^[0-9]+$

This pattern matches a string containing only digits.

---

23. POSIX-Style Regular Expressions

Older PHP versions provided POSIX regular-expression functions such as:

ereg()

eregi()

ereg_replace()

split()

These functions were deprecated and later removed from PHP.

Therefore, they should generally be treated as historical/legacy material, especially if they appear in older MCA textbooks.

Modern PHP applications should use PCRE/PCRE2 functions such as:

- preg_match()
- preg_match_all()
- preg_replace()
- preg_split()

---

24. Perl-Compatible Regular Expressions (PCRE)

PHP provides Perl-compatible regular expressions through the preg_* functions.

preg_match()

Checks whether a pattern matches a string.

<?php

ITM-MCA

```
$text = "MCA Student";
if (preg_match("/MCA/", $text)) {
   echo "Pattern Found";
}
?>
```

Output

Pattern Found

---

25. Regular Expression Pattern Syntax

A common PHP regex is written as:

/pattern/modifiers

Example:

/hello/i

Here:

- / → delimiter
- hello → pattern
- i → case-insensitive modifier

---

26. Common Regular Expression Symbols

Symbol	Meaning
^	Beginning of string
$	End of string
.	Any character except newline in common usage
*	Zero or more occurrences
+	One or more occurrences
?	Zero or one occurrence
{n}	Exactly n occurrences
{n,m}	Between n and m occurrences
[abc]	One character from a, b or c
[0-9]	Any digit
[A-Z]	Uppercase letter
[a-z]	Lowercase letter
\d	Digit
\w	Word character

ITM-MCA

Symbol Meaning

\s Whitespace

Example: Validate Digits

```
$number = "12345";
if (preg_match('/^[0-9]+$/', $number)) {
   echo "Valid number";
}
```

---

27. Validating an Email Using Regular Expression

A simple educational example:

```
$email = "student@example.com";
if (preg_match('/^[^@\s]+@[^@\s]+\.[^@\s]+$/', $email)) {
   echo "Valid email";
} else {
   echo "Invalid email";
}
```

For production applications, PHP's filter_var() is often preferable for basic email validation:

```
filter_var($email, FILTER_VALIDATE_EMAIL);
```

---

28. preg_match_all()

Used to find all occurrences of a pattern.

```
$text = "PHP is powerful. PHP is popular.";
preg_match_all('/PHP/', $text, $matches);
print_r($matches);
```

It stores all matches in the $matches array.

---

29. preg_replace()

Used to replace text matching a regular expression.

```
$text = "PHP 123";
$result = preg_replace('/[0-9]+/', 'XXX', $text);
echo $result;
```

Output

PHP XXX

---

30. preg_split()

Splits a string using a regular-expression pattern.

```
$text = "PHP,Java;Python";
```

ITM-MCA

```
$result = preg_split('/[,;]/', $text);
print_r($result);
```

Output:

```
Array
(
    [0] => PHP
    [1] => Java
    [2] => Python
)
```

PART D — ARRAYS IN PHP

31. Introduction to Arrays

An array is a data structure used to store multiple values in a single variable.

Instead of:

```
$name1 = "Amit";
$name2 = "Rahul";
$name3 = "Priya";
```

we can use:

```
$students = ["Amit", "Rahul", "Priya"];
```

Arrays are very important in PHP because they are used for:

- Lists
- Records
- Form data
- Database results
- Configuration data
- Multidimensional data

---

32. Indexed Arrays vs Associative Arrays

Indexed Array

An indexed array uses numeric keys.

```
$students = ["Amit", "Rahul", "Priya"];
```

Indexes:

0 → Amit

1 → Rahul

2 → Priya

Access:

```
echo $students[0];
```

ITM-MCA

Output:

Amit

Associative Array

An associative array uses named keys.

```
$student = [
   "name" => "Amit",
   "course" => "MCA",
   "age" => 23
];
```

Access:

```
echo $student["name"];
```

Output:

Amit

Difference

Indexed Array	Associative Array
Numeric keys	Named keys
$arr[0]	$arr["name"]
Suitable for lists	Suitable for records/data with labels

---

33. Identifying Elements of an Array

Individual array elements can be accessed using their keys.

Indexed Array

```
$colors = ["Red", "Green", "Blue"];
echo $colors[1];
```

Output:

Green

Associative Array

```
$student = [
   "name" => "Rahul",
   "age" => 24
];
echo $student["age"];
```

Output:

24

ITM-MCA

34. Storing Data in Arrays

Arrays can be created using [].

```
$marks = [80, 75, 90, 85];
```

Elements can be added:

```
$marks[] = 95;
```

Associative elements can be added:

```
$student["city"] = "Nagpur";
```

Using array_push()

```
$colors = ["Red", "Green"];
array_push($colors, "Blue", "Yellow");
print_r($colors);
```

---

35. Multidimensional Arrays

A multidimensional array is an array containing other arrays.

Example

```
$students = [
    ["Amit", 80],
    ["Rahul", 75],
    ["Priya", 90]
];
```

Access:

```
echo $students[0][0];
```

Output:

Amit

```
echo $students[1][1];
```

Output:

75

Associative Multidimensional Array

```
$students = [
    [
        "name" => "Amit",
        "marks" => 80
    ],
    [
        "name" => "Rahul",
        "marks" => 75
```

ITM-MCA

```
    ]
];
echo $students[0]["name"];
```

Output:

Amit

---

36. Extracting Multiple Values

PHP provides list() and array destructuring for extracting multiple values.

Example

```
$student = ["Amit", "MCA", 23];
[$name, $course, $age] = $student;
echo $name;
echo $course;
echo $age;
```

Output:

AmitMCA23

A clearer version:

```
echo "Name: $name<br>";
echo "Course: $course<br>";
echo "Age: $age";
```

---

37. Converting Between Arrays and Variables

extract()

extract() converts associative-array keys into variable names.

```
$student = [
    "name" => "Amit",
    "age" => 23
];
extract($student);
echo $name;
echo $age;
```

Output:

Amit23

Important

extract() should be used carefully when processing untrusted data because it can create or overwrite variables unexpectedly.

ITM-MCA

compact()

compact() performs the reverse operation.

```
$name = "Amit";
$age = 23;
$student = compact("name", "age");
print_r($student);
```

Result:

```
Array
(
   [name] => Amit
   [age] => 23
)
```

Difference

extract() → Array → Variables

compact() → Variables → Array

---

38. Traversing Arrays

Traversing means visiting each element of an array one by one.

Using for

```
$students = ["Amit", "Rahul", "Priya"];
for ($i = 0; $i < count($students); $i++) {
   echo $students[$i] . "<br>";
}
```

Using foreach

foreach is one of the most commonly used methods for traversing PHP arrays.

```
$students = ["Amit", "Rahul", "Priya"];

foreach ($students as $student) {
   echo $student . "<br>";
}
```

Associative Array

```
$student = [
   "name" => "Amit",
   "course" => "MCA",
   "age" => 23
];
```

ITM-MCA

```
foreach ($student as $key => $value) {
   echo "$key : $value<br>";
}
```

---

39. Sorting Arrays

PHP provides several built-in sorting functions.

sort()

Sorts indexed arrays in ascending order.

```
$numbers = [50, 10, 30, 20];
sort($numbers);
print_r($numbers);
```

Result:

10 20 30 50

---

rsort()

Sorts in descending order.

```
rsort($numbers);
```

---

asort()

Sorts an associative array by value while preserving keys.

```
$marks = [
   "Amit" => 80,
   "Rahul" => 70,
   "Priya" => 90
];
asort($marks);
print_r($marks);
```

---

ksort()

Sorts an associative array by key.

```
ksort($marks);
```

---

arsort()

Sorts associative arrays by value in descending order.

```
arsort($marks);
```

ITM-MCA

krsort()

Sorts associative arrays by key in descending order.

krsort($marks);

Sorting Summary

Function	Sorting
sort()	Values ascending; reindexes numeric keys
rsort()	Values descending; reindexes numeric keys
asort()	Values ascending; preserves keys
arsort()	Values descending; preserves keys
ksort()	Keys ascending
krsort()	Keys descending

---

40. Acting on Entire Arrays

PHP provides functions that operate on complete arrays.

count()

Returns the number of elements.

```
$numbers = [10, 20, 30, 40];
echo count($numbers);
```

Output:

4

---

in_array()

Checks whether a value exists.

```
$colors = ["Red", "Green", "Blue"];
if (in_array("Green", $colors)) {
   echo "Color found";
}
```

---

array_search()

Returns the key of a matching value.

```
$colors = ["Red", "Green", "Blue"];
$key = array_search("Green", $colors);
echo $key;
```

Output:

1

Use strict comparison when appropriate:

ITM-MCA

array_search("1", $array, true);

---

array_sum()

Calculates the sum.

$marks = [80, 70, 90];

echo array_sum($marks);

Output:

240

---

array_reverse()

Reverses an array.

$numbers = [1, 2, 3, 4];

print_r(array_reverse($numbers));

---

array_unique()

Removes duplicate values.

$numbers = [10, 20, 10, 30, 20];

$result = array_unique($numbers);

print_r($result);

---

41. Array Operations

Merging Arrays

array_merge() combines arrays.

$a = ["A", "B"];

$b = ["C", "D"];

$result = array_merge($a, $b);

print_r($result);

Output:

```
Array
(
    [0] => A
    [1] => B
    [2] => C
    [3] => D
)
```

ITM-MCA

Slicing an Array

array_slice() extracts a portion.

```
$numbers = [10, 20, 30, 40, 50];
$result = array_slice($numbers, 1, 3);
print_r($result);
```

Result:

20 30 40

Removing Elements

array_pop() removes the last element.

```
$numbers = [10, 20, 30];
array_pop($numbers);
print_r($numbers);
```

Result:

10 20

array_shift() removes the first element.

```
array_shift($numbers);
```

---

42. Using Arrays with Functions

Arrays can be passed to functions.

```
function calculateTotal($marks) {
    return array_sum($marks);
}
$marks = [80, 75, 90];
echo calculateTotal($marks);
```

Output:

245

---

43. array_map()

array_map() applies a function to every element of an array.

```
$numbers = [1, 2, 3, 4];
$squares = array_map(function($n) {
    return $n * $n;
}, $numbers);
print_r($squares);
```

Result:

1 4 9 16

ITM-MCA

Using an arrow function:

```
$squares = array_map(fn($n) => $n * $n, $numbers);
```

---

44. array_filter()

array_filter() keeps elements that satisfy a condition.

```
$numbers = [10, 15, 20, 25, 30];
$result = array_filter($numbers, function($n) {
   return $n > 20;
});
print_r($result);
```

Result:

25 30

---

45. array_reduce()

array_reduce() reduces an array to a single value.

Example: calculating a total.

```
$numbers = [10, 20, 30];
$total = array_reduce($numbers, function($carry, $value) {
   return $carry + $value;
}, 0);
echo $total;
```

Output:

60

This is particularly useful for understanding functional programming concepts in PHP.

---

46. Important Array Functions — Quick Reference

Function	Purpose
count()	Counts elements
sort()	Sorts values ascending
rsort()	Sorts values descending
asort()	Sorts associative array by value
arsort()	Reverse value sort
ksort()	Sorts by key
krsort()	Reverse key sort
array_push()	Adds elements at end

ITM-MCA

Function	Purpose
array_pop()	Removes last element
array_shift()	Removes first element
array_unshift()	Adds elements at beginning
array_merge()	Combines arrays
array_slice()	Extracts part of array
array_unique()	Removes duplicates
in_array()	Checks whether value exists
array_search()	Searches for value and returns key
array_sum()	Calculates sum
array_reverse()	Reverses array
array_map()	Applies function to each element
array_filter()	Filters elements
array_reduce()	Reduces array to one result

---

47. Integrated Example — Functions, Strings and Arrays

The following program combines the major concepts from this unit.

```
<?php
function processStudents($students) {
   foreach ($students as $student) {
      $name = trim($student["name"]);
      $marks = $student["marks"];
      $total = array_sum($marks);
      $average = $total / count($marks);
      echo "Student: " . htmlspecialchars($name) . "<br>";
      echo "Total Marks: " . $total . "<br>";
      echo "Average: " . $average . "<br>";
      if ($average >= 40) {
         echo "Result: Pass<br>";
      } else {
         echo "Result: Fail<br>";
      }
      echo "<hr>";
   }
}
```

ITM-MCA

```
$students = [
    [
        "name" => " Amit ",
        "marks" => [80, 75, 90]
    ],
    [
        "name" => "Rahul",
        "marks" => [60, 65, 70]
    ]
];
processStudents($students);
?>
```

Concepts Demonstrated

This single program demonstrates:

- Function definition
- Function parameters
- Function calling
- Associative arrays
- Multidimensional arrays
- foreach
- trim()
- htmlspecialchars()
- array_sum()
- count()
- Conditional statements
- Return-oriented program design

---

48. Conceptual Relationship Between the Topics

These topics are closely connected:

```
              PHP
               |
      -----------------------
     |         |         |
  Functions   Strings   Arrays
     |         |         |
 Parameters  Searching  Indexed
```

ITM-MCA

Return	Cleaning	Associative
Scope	Comparing	Multidimensional
Anonymous	Regex	Traversing
Variable	Encoding	Sorting
Functions		

For an MCA student, these concepts form an important foundation for modular programming and data processing in PHP.

---

49. Important Differences for Examination

Function vs Anonymous Function

Function	Anonymous Function
Has a name	Does not have a name
Can be called by its name	Usually stored in a variable or passed as callback
Suitable for reusable named operations	Useful for short/dynamic operations

Indexed vs Associative Array

Indexed	Associative
Numeric keys	Named keys
$a[0]	$a["name"]
Good for lists	Good for records

echo vs return

echo	return
Displays output	Sends value back
Primarily for output	Used to provide a result to caller
Cannot be assigned as function result in the same way	Returned value can be stored and reused

POSIX Regex vs PCRE

POSIX Style	PCRE
Older PHP regex approach	Modern PHP regex approach
ereg() etc.	preg_match() etc.
Removed from modern PHP	Currently used
Mostly historical	Recommended for PHP regex work

IITM-MCA

Important MCA Examination Questions

Short/Long Answer Questions

1. Explain defining and calling functions in PHP with examples.
2. Explain local, global, and static variable scope.
3. Explain function parameters and arguments.
4. Explain pass-by-value and pass-by-reference.
5. Explain return values with suitable examples.
6. Explain variable functions and anonymous functions.
7. Explain different methods of quoting and printing strings.
8. Explain string cleaning, encoding, and escaping.
9. Explain different PHP string manipulation functions.
10. Explain regular expressions and their applications.
11. Explain POSIX-style and Perl-compatible regular expressions.
12. Explain preg_match(), preg_match_all(), preg_replace(), and preg_split().
13. Differentiate indexed and associative arrays.
14. Explain multidimensional arrays with examples.
15. Explain different methods of traversing arrays.
16. Explain array sorting functions in PHP.
17. Explain array_map(), array_filter(), and array_reduce().
18. Explain methods for converting between arrays and variables.
19. Explain commonly used PHP array functions.
20. Write a PHP program demonstrating functions, strings, regular expressions, and arrays together.

IITM-MCA